

```
import tensorflow as tf
from sklearn.preprocessing import StandardScaler
from sklearn import metrics
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")
```

```
boston = tf.keras.datasets.boston_housing
(x_train, y_train), (x_test, y_test) = boston.load_data()

print(x_train.shape, y_train.shape)
print(x_test.shape, y_test.shape)
```

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/boston_housing.npz
 57026/57026 ————— 0s 0us/step
 (404, 13) (404,)
 (102, 13) (102,)

```
scaler = StandardScaler()

x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)

y_train_scaled = scaler.fit_transform(y_train.reshape(-1, 1))
y_test_scaled = scaler.transform(y_test.reshape(-1, 1))
```

```
model = tf.keras.models.Sequential([
    tf.keras.layers.Input(shape=(13,), name='input-layer'),
    tf.keras.layers.Dense(100, name='hidden-layer-2'),
    tf.keras.layers.BatchNormalization(name='hidden-layer-3'),
    tf.keras.layers.Dense(50, name='hidden-layer-4'),
    tf.keras.layers.Dense(1, name='output-layer')
])
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
hidden-layer-2 (Dense)	(None, 100)	1,400
hidden-layer-3 (BatchNormalization)	(None, 100)	400
hidden-layer-4 (Dense)	(None, 50)	5,050
output-layer (Dense)	(None, 1)	51

Total params: 6,901 (26.96 KB)

Trainable params: 6,701 (26.18 KB)

```
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)
```

```
history = model.fit(
    x_train_scaled, y_train_scaled,
    batch_size=32,
    epochs=20,
    validation_data=(x_test_scaled, y_test_scaled)
)
```

```
Epoch 1/20
13/13 ————— 3s 43ms/step - loss: 1.5347 - mae: 0.8707 - val_loss: 0.7266 - val_mae: 0.6306
Epoch 2/20
13/13 ————— 0s 14ms/step - loss: 0.6156 - mae: 0.5890 - val_loss: 0.7624 - val_mae: 0.6415
Epoch 3/20
13/13 ————— 0s 16ms/step - loss: 0.4359 - mae: 0.4925 - val_loss: 0.7682 - val_mae: 0.6282
Epoch 4/20
13/13 ————— 0s 22ms/step - loss: 0.4321 - mae: 0.4803 - val_loss: 0.8007 - val_mae: 0.6608
Epoch 5/20
13/13 ————— 0s 13ms/step - loss: 0.3996 - mae: 0.4652 - val_loss: 0.5745 - val_mae: 0.5275
Epoch 6/20
```

```

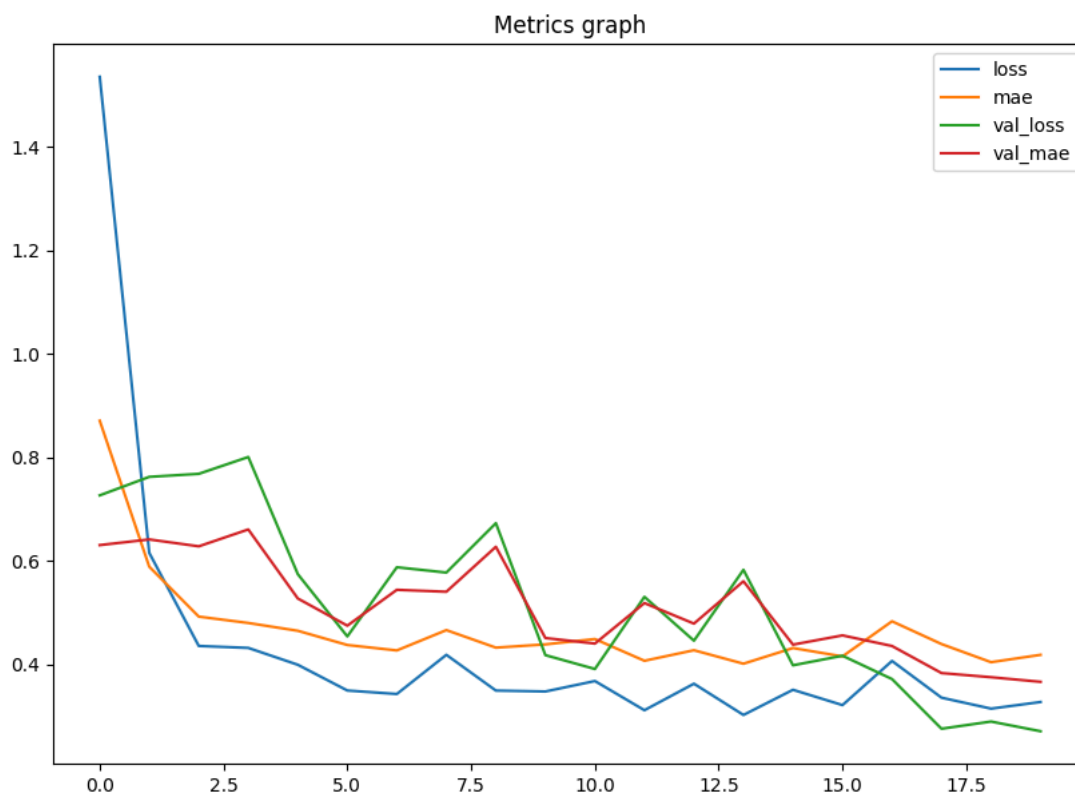
13/13 ————— 0s 11ms/step - loss: 0.3499 - mae: 0.4377 - val_loss: 0.4543 - val_mae: 0.4751
Epoch 7/20
13/13 ————— 0s 11ms/step - loss: 0.3431 - mae: 0.4272 - val_loss: 0.5879 - val_mae: 0.5443
Epoch 8/20
13/13 ————— 0s 13ms/step - loss: 0.4188 - mae: 0.4665 - val_loss: 0.5776 - val_mae: 0.5405
Epoch 9/20
13/13 ————— 0s 11ms/step - loss: 0.3498 - mae: 0.4326 - val_loss: 0.6730 - val_mae: 0.6274
Epoch 10/20
13/13 ————— 0s 13ms/step - loss: 0.3480 - mae: 0.4389 - val_loss: 0.4182 - val_mae: 0.4512
Epoch 11/20
13/13 ————— 0s 13ms/step - loss: 0.3682 - mae: 0.4492 - val_loss: 0.3914 - val_mae: 0.4405
Epoch 12/20
13/13 ————— 0s 11ms/step - loss: 0.3118 - mae: 0.4073 - val_loss: 0.5307 - val_mae: 0.5187
Epoch 13/20
13/13 ————— 0s 14ms/step - loss: 0.3630 - mae: 0.4276 - val_loss: 0.4460 - val_mae: 0.4790
Epoch 14/20
13/13 ————— 0s 14ms/step - loss: 0.3027 - mae: 0.4016 - val_loss: 0.5829 - val_mae: 0.5608
Epoch 15/20
13/13 ————— 0s 8ms/step - loss: 0.3513 - mae: 0.4319 - val_loss: 0.3985 - val_mae: 0.4385
Epoch 16/20
13/13 ————— 0s 7ms/step - loss: 0.3217 - mae: 0.4157 - val_loss: 0.4170 - val_mae: 0.4563
Epoch 17/20
13/13 ————— 0s 8ms/step - loss: 0.4069 - mae: 0.4835 - val_loss: 0.3720 - val_mae: 0.4358
Epoch 18/20
13/13 ————— 0s 8ms/step - loss: 0.3360 - mae: 0.4396 - val_loss: 0.2761 - val_mae: 0.3835
Epoch 19/20
13/13 ————— 0s 8ms/step - loss: 0.3149 - mae: 0.4045 - val_loss: 0.2899 - val_mae: 0.3755
Epoch 20/20
13/13 ————— 0s 8ms/step - loss: 0.3277 - mae: 0.4187 - val_loss: 0.2712 - val_mae: 0.3666

```

```

pd.DataFrame(history.history).plot(figsize=(10,7))
plt.title("Metrics graph")
plt.show()

```



```

y_pred_scaled = model.predict(x_test_scaled)
y_pred = scaler.inverse_transform(y_pred_scaled)

```

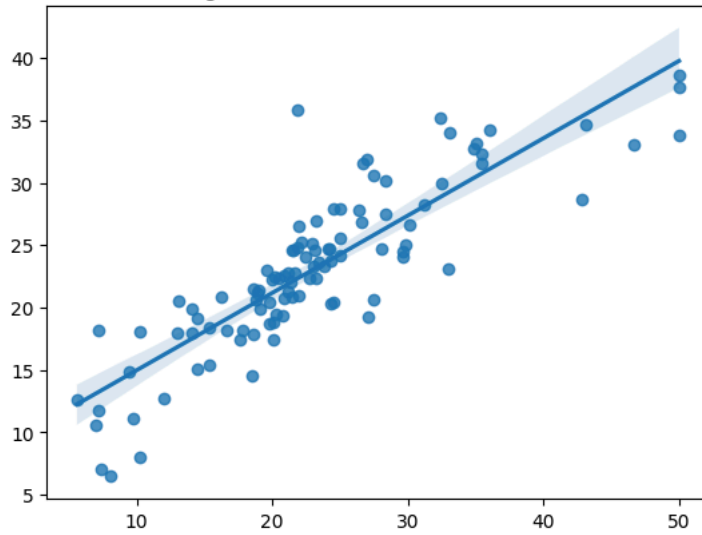
```
4/4 ————— 0s 21ms/step
```

```

sns.regplot(x=y_test, y=y_pred)
plt.title("Regression Line for Predicted values")
plt.show()

```

Regression Line for Predicted values



```
def regression_metrics_display(y_test, y_pred):  
    print("MAE is", metrics.mean_absolute_error(y_test, y_pred))  
    print("MSE is", metrics.mean_squared_error(y_test, y_pred))  
    print("R2 score is", metrics.r2_score(y_test, y_pred))
```

```
regression_metrics_display(y_test, y_pred)
```

```
MAE is 3.3727832850287944  
MSE is 22.95220221685956  
R2 score is 0.724277497677198
```

Start coding or [generate](#) with AI.